

LLM 部署在硬件测试中的应用探索

一、痛点分析

1.1 GPU 业务场景验证不足，外场故障频发

eg1:

山西电信 R6500G5 GPU 掉卡问题

故障现象：6500G5+Nvidia-A800，所内测试和生产测试均有 GPU 压测，但未发现有 GPU 掉卡，客户现场跑模型业务出现掉卡。

故障定位：硬件和电源同步排查 12V 电源问题。目前排除过温导致故障，现在挂示波器排查电流问题。增加飞线进行测试，电源抓抓到 12V 电源跌落，目前电源分析中。目前定位 efuse 保险丝限流导致掉卡，生产已紧急停线，电源平台 13 日前完成替换为 81292 可行性评估。

解决方案：更换 33 台整机的主板。

eg2:

008.IEC-653005浦江R6900G5+BR掉卡问题

由 张玉航10279235 创建，最终由 王罡10039077 修改于 2025-05-10 16:08:04 3 0 0

责任领域	外购件		
产品型号	R6900G5		
创建时间	2025-01-20		
计划解决日期	2025-01-26		
处理时长			
故障现象	64台机器，现场6台掉卡（不确认），跑模型过程中掉卡，有1台跟踪槽位走。另外5台交换OAM后，跑2天，暂时还未查看结果；前期BR自己在排查，19日研发才收到故障通知。		
故障级别	紧急（日报）		
处理状态	方案规划中		
预警状态	正常推进		
方案分类	解决根因		
责任人	@郑寒冰10295959		
交付团队跟踪人	@杜猛10317408		
相关人	@陈泉10217299	@付涛10086294	@韩栋10124986
责任部长	@罗卉00041759		

惠州腾讯R69G5-1784台设备-DOA看板

由 吴亦杰6657001847 创建, 最终由 吴亦杰6657001847 修改于 2025-09-02 09:54:56 3 0 0

一句话总结:

腾讯R6900G5服务器1784台, 目前DOA总计81单 (含二次报障8单), DOA率4.54% (81/1784), 当前已关闭80单, 正在处理1单

ps: 由于腾讯现场工单时效24小时, 按照恢复为主方式处理

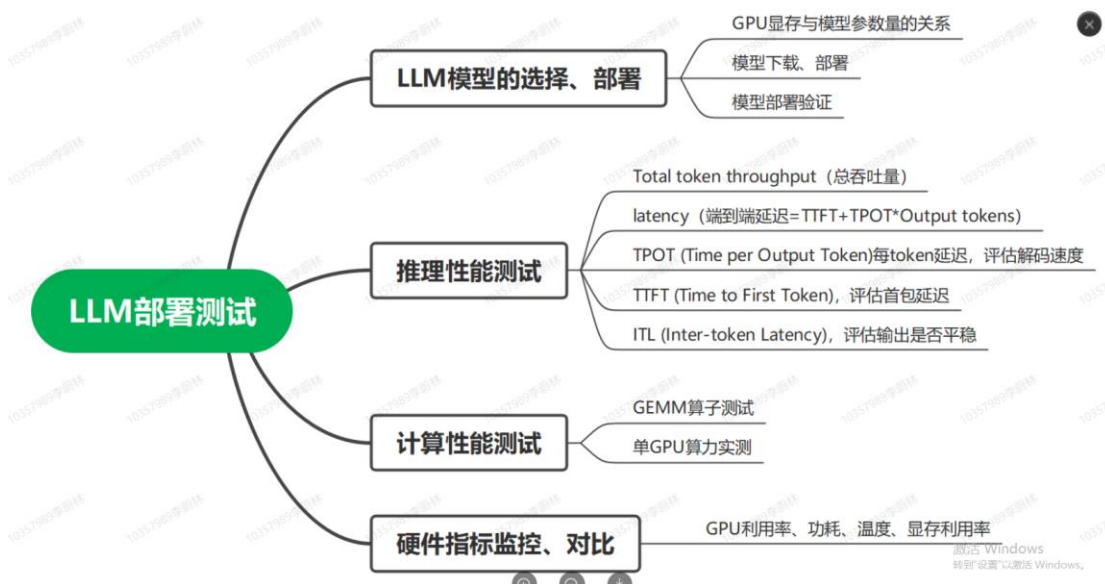


由上述 2 个外场案例可以看出, 在所内经过研发测试、生产测试, 但外场在做模型业务时仍有故障泄露, 究其原因, 1 是研发测试和生产测试没有部署模型相关的测试, 2 是研发测试 GPU 压测一般都选择通用压力测试工具如 GPU Burn 等, 虽然 GPU 利用率和功耗可以拉满, 但是不管是压力、利用率还是功耗都是平稳的曲线, 不能有效模拟外场业务场景动态负载的场景

1.2 研发≠客户, 不了解客户的应用需求

服务器不同于无线产品, 服务器的客户众多, 且不同客户做的业务也不同, 目前我们还只是设备生产厂商, 加之客户对于自己的模型业务也十分保密, 因此我们对于 GPU 的使用知之甚少。

二、应对方案



三、大模型选择与部署

3.1 模型选择与 GPU 显存

部署大型语言模型（LLM）不仅要选择合适的模型，还需确保配备合适的硬件来高效运行它。在这一过程中，一个关键因素在于 GPU 显存——其容量将直接影响到模型在生产环境中的性能、速度及可行性。

无论大家使用的是拥有数十亿参数的模型还是更复杂的架构，了解所需 GPU 显存容量至关重要——它不仅能避免系统崩溃、运行卡顿，更能减少不必要的资源浪费。为 LLM 提供服务需要 GPU 显存来存储模型权重、中间计算和额外的开销，以实现高效处理。当前业界存在一种简单直观的显存估算方法，一个计算公式，可以帮助估算 GPU 所需的显存大小：

$$M = \left(\frac{P \times 4B}{\frac{32}{Q}} \right) \times 1.2$$

上述公式中：

M 表示所需的 GPU 显存，单位为千兆字节 (GB)

P 表示模型中的参数量

4B 表示每个参数占用 4 个字节

Q 表示加载模型时每个参数的位数（例如 FP16 为 16 位）

1.2 = 20% 的额外预留系数

要高效地为模型提供服务，需要在显存中存储以下内容：

模型参数 - 这是模型的通过学习得到的权重。

中间计算 - 用于推理计算的临时内存。

开销 - 用于高效数据移动和并行处理的额外内存。

eg:

7B 参数模型需要多少内存？

做以下假设：

P = 70 亿 (7B) 个参数

Q = 16 比特精度 (FP16)

将这些设置代入公式：

因此，要为 FP16 精度的 7B 参数模型提供服务，至少需要 16.8GB GPU 显存。

但是，如果使用量化的 8 比特模型（Q = 8），所需的显存将减半，约为 8.4GB。

为什么需要额外的 1.2 预留系数？

我们公式中的 1.2 系数（20% 额外系数）并不只是一个任意的数字，它在确保顺利进行模型推理方面起着至关重要的作用。但究竟为什么要加上 20%，而不是 50% 或其他数字呢？让我们来分析一下。

内存碎片

当加载模型时，GPU 显存会按块分配，但这些显存块很少能完美紧密排列。由于内存碎片化的存在，部分显存会被浪费，这意味着实际可用显存容量会略低于显卡标称的 VRAM 总量。为此预留 20% 的缓冲空间，正是为了抵消这种内存管理上的效率损耗。

中间结果

在模型推理过程中，GPU 不仅要存储权重参数，还会动态生成被称为激活值（activations）的中间计算结果。这些激活值需要占用额外显存，且模型参数量越大，此类中间数据消耗的内存就越多。

GPU 资源优化

现代 GPU 会保留部分显存用于系统级操作、内核执行及 CUDA 相关进程。若将全部显存分配给模型，可能导致系统不稳定甚至崩溃。额外预留的 20% 显存空间，正是为了确保 GPU 在执行这些后台任务时拥有足够的显存余量，避免因资源耗尽而引发故障。

为什么不是 50%？

在多数应用场景中，预留 50% 的缓冲空间未免过于保守，容易导致 GPU 资源浪费。20% 的黄金法则巧妙地在系统稳定性与模型可用显存之间取得平衡。然而，若您运行的是处理需求极为复杂的超大规模模型（例如包含视觉组件的多模态模型），则可能需要更高的缓冲比例。

3.2 模型下载及部署

3.2.1 使用 modelscope 下载模型

模型下载地址：<https://modelscope.cn/models>，在 ModelScope 社区下载模型的三种方式：

- 1、使用命令行工具下载
- 2、使用 SDK 下载
- 3、通过 Git 下载

命令示例（以 Qwen2-7B）模型为例

下载整个模型 repo（到默认 cache 地址）#

```
modelscope download --model 'Qwen/Qwen2-7b'
```

下载整个模型 repo 到指定目录#

```
modelscope download --model 'Qwen/Qwen2-7b' --local_dir 'path/to/dir'
```

指定下载单个文件(以'tokenizer.json'文件为例)#

```
modelscope download --model 'Qwen/Qwen2-7b' tokenizer.json
```

指定下载多个文件#

```
modelscope download --model 'Qwen/Qwen2-7b' tokenizer.json config.json
```

指定下载某些文件#

```
modelscope download --model 'Qwen/Qwen2-7b' --include '*.safetensors'
```

```
(modelscope) [root@localhost home]# cd modelScope/
(modelscope) [root@localhost modelScope]# ls
Miniconda3-latest-Linux-x86_64.sh  model-dir  outputs
(modelscope) [root@localhost modelScope]#
(modelscope) [root@localhost modelScope]#
(modelscope) [root@localhost modelScope]#
(modelscope) [root@localhost modelScope]# cd model-dir/
(modelscope) [root@localhost model-dir]# ls
deepseek-R1-Distill  Llama3.2  outputs  Qwen2.5-0.5B
(modelscope) [root@localhost model-dir]#
```

3.2.2 使用 vllm 部署模型

1、推理即使用模型用输入获得输出的过程，部署是将模型发布到恒定运行的环境中推理的过程。一般来说，LLM 的推理可以直接使用 PyTorch 代码、使用 VLLM/XInference/FastChat 等框架，也可以使用 llama.cpp/chatglm.cpp/qwen.cpp

等 c++推理框架。

2、VLLM 框架

网址: <https://github.com/vllm-project/vllm>

vLLM 是一个开源的大模型推理加速框架,通过 PagedAttention 高效地管理 attention 中缓存的张量,实现了比 HuggingFace Transformers 高 14-24 倍的吞吐量。PagedAttention 是 vLLM 的核心技术,它解决了 LLM 服务中内存的瓶颈问题。传统的注意力算法在自回归解码过程中,需要将所有输入 Token 的注意力键和值张量存储在 GPU 内存中,以生成下一个 Token。这些缓存的键和值张量通常被称为 KV 缓存。

3、实际部署

环境: R2350 G6+4 L20GPU

步骤 1: 环境准备,安装依赖包。使用 conda 安装虚拟环境,安装 python 包(关键库: modelscope、nvidia-cuda、torch、vllm 等)

步骤 2: 进入虚拟环境,使用 vllm 加载下载好的模型:

CUDA_VISIBLE_DEVICES=1, 指定使用某个 GPU 卡,一般默认从 0 开始,可以使用 nvidia-smi 查看 GPU 信息。

eg:

```
CUDA_VISIBLE_DEVICES=1          vllm          serve
/home/modelScope/model-dir/Qwen2.5-0.5B --tensor_parallel_size 1
--served-model-name "Qwen2.5-0.5B-Instruct" --max-num-seqs 256
--max-model-len=4096 --host 10.236.75.106 --port 8001 &
```

```
Capturing CUDA graphs (mixed prefill-decode, PIECEWISE): 100% | 51/51 [00:00<00:00, 100.92it/s]
Capturing CUDA graphs (decode, FULL): 100% | 35/35 [00:00<00:00, 118.41it/s]
(EngineCore_DPO pid=16574) INFO 05-28 14:50:29 [core.py:278] Graph capturing finished in 2 secs, took 0.43 GiB
(EngineCore_DPO pid=16574) INFO 05-28 14:50:29 [core.py:278] init engine (profile, create kv cache, warmup model) took 6.53 seconds
(EngineCore_DPO pid=16574) INFO 05-28 14:50:30 [vllm.py:889] Asynchronous scheduling is enabled.
(APIServer pid=16304) INFO 05-28 14:50:30 [api_server.py:481] Supported tasks: ['generate']
(APIServer pid=16304) WARNING 05-28 14:50:30 [model.py:1358] Default vLLM sampling parameters have been overridden by the model's 'generation_config.json': {'repetition_penalty': 1.1, 'temperature': 0.7, 'top_k': 20, 'top_p': 0.8}. If this is not intended, please relaunch vLLM instance with '--generation-config vllm'.
(APIServer pid=16304) INFO 05-28 14:50:30 [serving.py:188] Warming up chat template processing...
(APIServer pid=16304) INFO 05-28 14:50:30 [hf.py:218] Detected the chat template content format to be 'string'. You can set '--chat-template-content-format' to override this.
(APIServer pid=16304) INFO 05-28 14:50:30 [serving.py:213] Chat template warmup completed in 127.8ms
(APIServer pid=16304) INFO 05-28 14:50:30 [api_server.py:486] Starting vLLM API server 0 on http://10.236.75.106:8001
(APIServer pid=16304) INFO 05-28 14:50:30 [launcher.py:38] Available routes are:
(APIServer pid=16304) INFO 05-28 14:50:30 [launcher.py:47] Route: /openapi.json, Methods: GET, HEAD
(APIServer pid=16304) INFO 05-28 14:50:30 [launcher.py:47] Route: /docs, Methods: GET, HEAD
(APIServer pid=16304) INFO 05-28 14:50:30 [launcher.py:47] Route: /docs/oauth2-redirect, Methods: GET, HEAD
(APIServer pid=16304) INFO 05-28 14:50:30 [launcher.py:47] Route: /redoc, Methods: GET, HEAD
(APIServer pid=16304) INFO 05-28 14:50:30 [launcher.py:47] Route: /load, Methods: GET
(APIServer pid=16304) INFO 05-28 14:50:30 [launcher.py:47] Route: /version, Methods: GET
(APIServer pid=16304) INFO 05-28 14:50:30 [launcher.py:47] Route: /scale-elastic-ep, Methods: POST
```

```
(modelscope) [root@localhost model-dir]# nvidia-smi
Thu May 28 14:51:44 2026
```

NVIDIA-SMI 560.35.03			Driver Version: 560.35.03			CUDA Version: 12.6		
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute M.	MIG M.
0	NVIDIA L20							
N/A	40C	P8	27W / 350W	00000000:41:00:0	Off	0%	Default	0
				1MiB / 46068MiB			N/A	
1	NVIDIA L20							
N/A	51C	P0	86W / 350W	00000000:61:00:0	Off	0%	Default	0
				41801MiB / 46068MiB			N/A	
2	NVIDIA L20							
N/A	42C	P8	26W / 350W	00000000:81:00:0	Off	0%	Default	0
				1MiB / 46068MiB			N/A	
3	NVIDIA L20							
N/A	41C	P8	27W / 350W	00000000:A1:00:0	Off	0%	Default	0
				1MiB / 46068MiB			N/A	

Processes:							
GPU	GI	CI	PID	Type	Process name		GPU Memory Usage
ID	ID	ID					
1	N/A	N/A	16574	C	VLLM::EngineCore		41792MiB

3.2 模型部署验证

```
13 # --- 配置 ---
14 API_BASE_URL = "http://10.236.75.106:8001/v1"
15 PROMPT = "你好，你是谁？" # 用短Prompt测试更灵敏
16 PROMPT_LIST = [
17     "你是谁？你的模型名称全称是什么？",
18     "请用Python写一个快速排序算法。",
19     "解释一下什么是量子纠缠。",
20     "北京有什么好玩的地方？",
21     "如何制作一杯手冲咖啡？",
22     "请帮我写一首关于秋天的诗。",
23     "什么是微服务架构？它有什么优缺点？",
24     "推荐几本科幻小说。",
25     "如何提高英语口语水平？",
26     "请解释一下“显卡烤机刀”原理。",
27     "以太坊和比特币有什么区别？"
```

Q: 以太坊和比特币有什么区别？

A: 以太坊（Ethereum）和比特币（Bitcoin）是两种不同的数字货币，它们在许多方面都有显著的区别：

- 交易处理速度：以太坊的交易处理时间通常比比特币快得多。以太坊使用的是基于区块链技术的智能合约系统，这使得它能够在很短的时间内完成复杂的金融交易。
- 硬分叉事件：比特币是一个开源项目，其代码可以被修改或更改，导致新的分支出现。这些新分支被称为“硬分叉”。相比之下，以太坊是一个由多个独立团队共同维护的分布式账本系统，因此没有硬分叉的风险。
- 使用场景：比特币主要用于加密货币市场，包括挖矿、投资和日常消费等；而以太坊则更广泛地应用于区块链应用，如智能合约、去中心化金融服务等。
- 共识算法：以太坊采用的共识算法是基于权益证明（Proof of Stake），而比特币采用的是基于工作量证明（Workload Proof of Stake）。这意味着以太坊的交易处理速度更快，但可能会产生更高的能耗成本。
- 风险控制：以太坊面临的安全风险较高，因为其开发和维护依赖于多方合作，并且存在黑客攻击的风险。而比特币由于其公开性，更容易受到网络攻击的影响。

总的来说，以太坊和比特币各有优劣，用户可以根据自己的需求选择适合的数字货币进行交易。

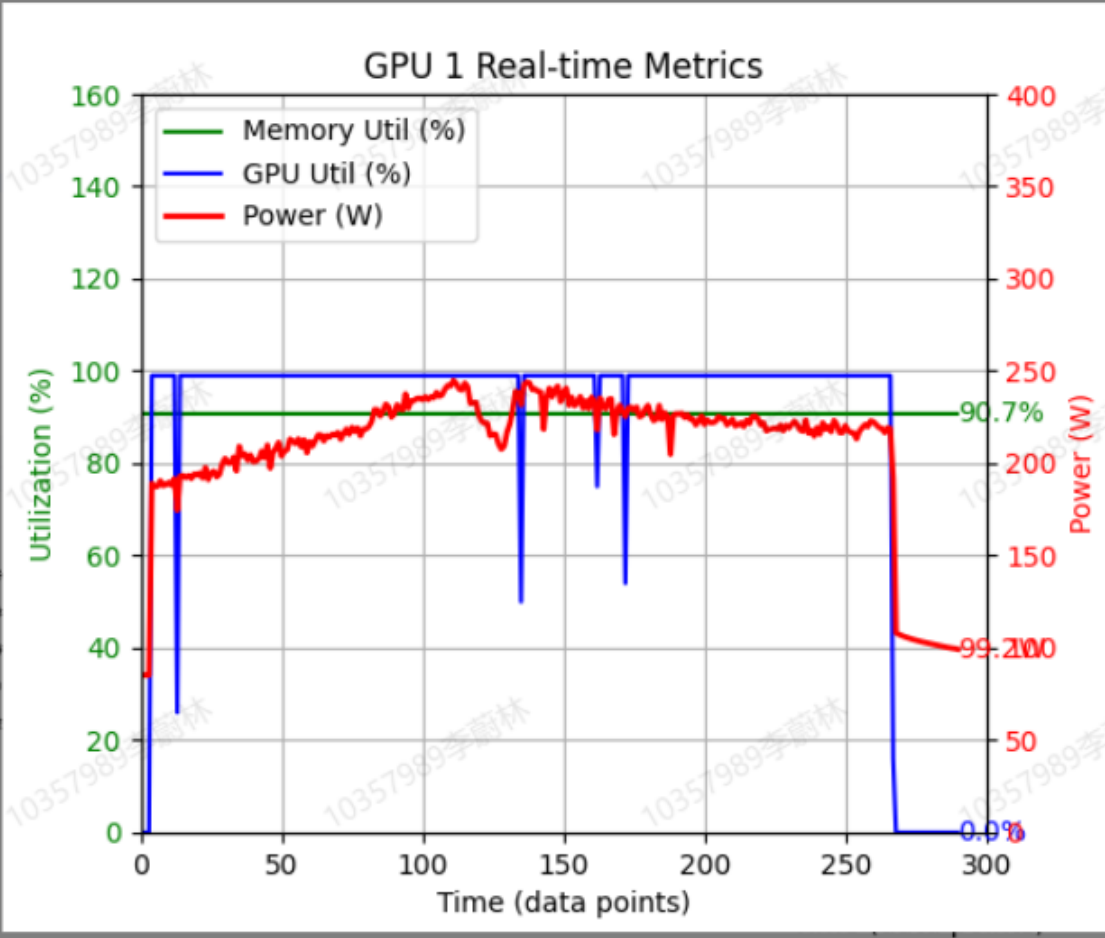
Q: 你是谁？你的模型名称全称是什么？

A: 我是由阿里云开发的超大规模语言模型，我叫通义千问。我的模型名称全称为通义千问，是阿里巴巴集团与阿里云联合推出的一款大型预训练模型。

Q: 北京有什么好玩的地方？

A: 北京，这座历史悠久、文化灿烂的城市，拥有众多吸引游客的景点和活动。以下是一些在北京值得一游的热门地点：

- 长城：作为中国最著名的旅游景点之一，长城是中国历史上的军事防御工程，也是世界文化遗产。
- 故宫：位于北京市中心，是明清两代的皇宫，拥有丰富的历史文化价值。



四、推理性能验证

4.1 推理指标解读

指标	英文名称	解释
首次生成token时间	TTFT (Time to First Token)	从发送请求到生成第一个token的时间（以秒为单位），评估首包延时
输出token间时延	ITL (Inter-token Latency)	生成每个输出token间隔时间（以秒为单位），评估输出是否平稳
每token延迟	TPOT (Time per Output Token)	生成每个输出token所需的时间（不包含首token，以秒为单位），评估解码速度
端到端延迟时间	Latency	从发送请求到接收完整响应的时间（以秒为单位）：TTFT + TPOT * Output tokens
输入token数	Input tokens	请求中输入的token数量
输出token数	Output tokens	响应中生成的token数量
输出吞吐量	Output Throughput	每秒输出的token数量：输出tokens / 端到端延时
总吞吐量	Total throughput	每秒处理的token数量：(输入tokens + 输出tokens) / 端到端延时

指标	定义	统计意义
Mean (平均值)	所有请求的 TTFT 总和除以请求数量	反映整体响应时间的“平均水平”，但易受极值影响
Median (中位数)	将所有 TTFT 值从小到大排序后位于中间位置的值	表示一半请求快于该值，一半慢于该值，抗异常值干扰强
P99 (第99百分位数)	99% 的请求的 TTFT 都小于等于该值	反映系统在最差情况下的尾部延迟 (Tail Latency)，体现用户体验下限

4.2 推理实测分析

```
(APIServer pid=21283) INFO: 10.236.75.106:59058 - "GET /metrics HTTP/1.1" 200 OK
tip: install termplotlib and gnuplot to plot the metrics
===== Serving Benchmark Result =====
Successful requests: 5000
Failed requests: 0
Maximum request concurrency: 50
Benchmark duration (s): 78.01
Total input tokens: 500000
Total generated tokens: 500000
Request throughput (req/s): 64.09
Output token throughput (tok/s): 6409.32
Peak output token throughput (tok/s): 9151.00
Peak concurrent requests: 150.00
Total token throughput (tok/s): 12818.64
-----Time to First Token-----
Mean TTFT (ms): 32.09
Median TTFT (ms): 32.91
P99 TTFT (ms): 50.07
-----Time per Output Token (excl. 1st token)-----
Mean TPOT (ms): 7.50
Median TPOT (ms): 5.98
P99 TPOT (ms): 13.35
-----Inter-token Latency-----
Mean ITL (ms): 7.43
Median ITL (ms): 5.99
P99 ITL (ms): 14.04
=====
(modelscope) [root@localhost ~]# (APIServer pid=21283) INFO 05-28 16:12:25 [loggers.py:259] Engine 000: Avg prompt throughput: 16.6 tokens/s, Avg generation throughput: 1812.2 tokens/s, Running: 0 reqs, GPU KV cache usage: 0.0%, Prefix cache hit rate: 99.2%
(APIServer pid=21283) INFO 05-28 16:12:35 [loggers.py:259] Engine 000: Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 0.0 tokens/s, Running: 0 reqs, Waiting: 0 reqs, GPU KV cache usage: 0.0%, Prefix cache hit rate: 99.2%
```

五、计算性能验证

5.1 计算性能指标解读

简称	定义	解读
TOPS	Tera Operations Per Second	每秒执行 10^{12} 次操作，通常指整数或定点运算（如INT8），广泛用于衡量边缘计算设备、AI推理芯片的算力。
TFLOPS	Tera Floating Point Operations Per Second	每秒执行 10^{12} 次浮点运算，是衡量高性能计算系统算力的核心单位之一，常用于评估GPU、AI加速芯片及数据中心的计算能力。1 PFLOPS = 1000 TFLOPS = 1,000,000 GFLOPS = 1,000,000,000 MFLOPS。
PFLOPS	Peta Floating Point Operations Per Second	每秒执行 10^{15} 次浮点运算，是衡量高性能计算系统算力的核心指标，常用于评估超级计算机、AI训练集群等大规模计算系统的性能。
FP8	8位浮点数	使用8位二进制存储，分为两种格式：E4M3（4位指数、3位尾数）和E5M2（5位指数、2位尾数）。内存占用仅为FP16的一半，显著降低带宽需求，适用于大规模AI模型的高效推理。适用于对延迟和功耗敏感的大模型推理，如端侧部署、实时翻译等。
FP16	半精度浮点数	使用16位二进制存储，包含1位符号位、5位指数位和10位尾数位。相比FP32（单精度），内存占用减少50%，计算速度更快，广泛用于深度学习训练与推理。适合大多数深度学习训练任务，尤其是需要较高数值精度的场景。FP16算力值可通过“核心数 × 频率 × 每周期操作数”估算理论峰值，但实际性能取决于内存带宽、并行效率和软件优化。
FP32	32位单精度浮点数，Full Precision Floating Point	是深度学习和高性能计算中广泛使用的一种数值格式，具有较高的精度和动态范围，常用于模型训练和高精度科学计算。
INT8	8位整型	一种低精度数据格式，使用8位二进制表示整数（范围：-128 到 127），常用于深度学习模型的量化推理。相比FP32（32位浮点），内存占用减少75%，计算效率显著提升。在AI推理中广泛用于边缘设备、服务器端大规模部署等场景。

5.2 GEMM 算子解析

1、GEMM（General Matrix Multiply，通用矩阵乘法）是深度学习和高性能计算中的核心运算之一，其基本形式为：

$$C = \alpha \cdot A \times B + \beta \cdot C$$

其中：

AA、BB、CC 是矩阵；

α 、 β 是标量系数；

当 $\alpha=1$ 、 $\beta=0$ 时，简化为标准矩阵乘法 $C=A \times B$ 。

2、GEMM 的关键特性

计算密集：GEMM 操作在神经网络中占比极高，尤其在全连接层、卷积层（通过 im2col 转换）等结构中广泛应用 38。

硬件友好：现代 GPU（如 NVIDIA 的 Tensor Core）、TPU 等加速器均针对 GEMM 进行了专门优化 11。

性能指标：常被用作衡量 AI 加速器算力的标准基准（如 TFLOPS）

3、主要应用场景

深度学习：

全连接层直接由 GEMM 实现；

卷积层可通过 im2col 或 Implicit GEMM 转换为 GEMM 1011。

科学计算：物理模拟、气象预测、工程仿真等涉及大规模线性代数运算的领域 8。

混合专家模型（MoE）：GEMM 用于动态路由和专家计算，对低精度（如 FP8）支持要求高。

4、矩阵计算基础

矩阵乘法 $A[mn]XB[np]=C[mp]$, 运算次数= $m \times p \times n$ (乘法运算)+ $m \times p \times (n-1)$ 次加法运算。即一个 m 行 n 列的矩阵和一个 n 行 p 列的矩阵相乘, 其测试结果需要做 $m \times p \times n$ 次乘法运算和 $m \times p \times (n-1)$ 次加法运算。

两个矩阵的乘法仅当第一个矩阵 A 的列数和另一个矩阵 B 的行数相等时才能定义。如 A 是 $m \times n$ 矩阵和 B 是 $n \times p$ 矩阵, 它们的乘积 C 是一个 $m \times p$ 矩阵

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{in}b_{nj} = \sum_{r=1}^n a_{ir}b_{rj}$$

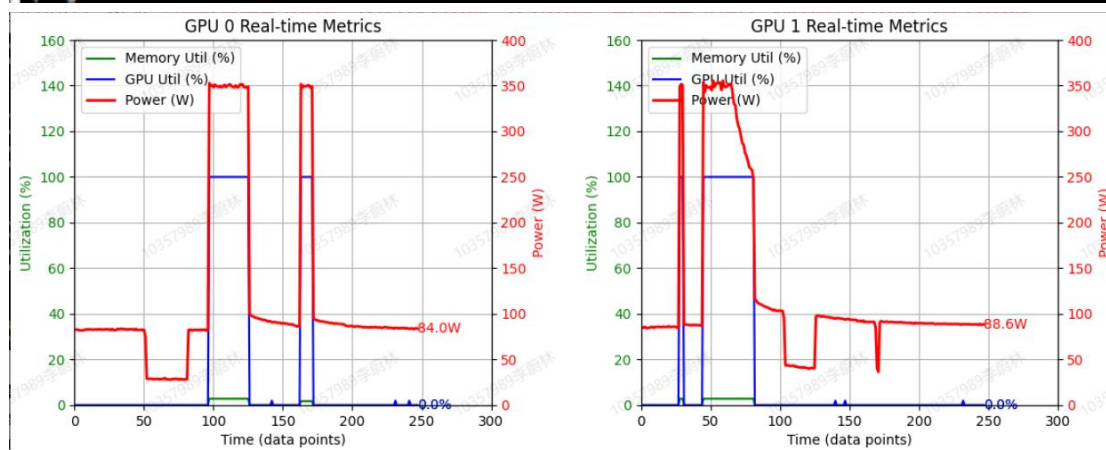
并将此乘积记为 $C=AB$, 例如:

$$\begin{bmatrix} 1 & 0 & 2 \\ -1 & 3 & 1 \end{bmatrix} \times \begin{bmatrix} 3 & 1 \\ 2 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} (1 \times 3 + 0 \times 2 + 2 \times 1) & (1 \times 1 + 0 \times 1 + 2 \times 0) \\ (-1 \times 3 + 3 \times 2 + 1 \times 1) & (-1 \times 1 + 3 \times 1 + 1 \times 0) \end{bmatrix} = \begin{bmatrix} 5 & 1 \\ 4 & 2 \end{bmatrix}$$

5.3 算力实测

实测: 配置: R2350 G6+4*L20, 实测单 GPU 算力 (FP32): 34312.61 GFLOPS=34.3TFLOPS, 单 GPU 算力 (FP16): 106590.53 GFLOPS=106.6TFLOPS,

```
(modelscope) [root@localhost Torch]# CUDA_VISIBLE_DEVICES=1 python GEMM.py
设备: cuda
数据类型: torch.float32
矩阵维度: 9216x9216 * 9216x9216 = 9216x9216
测试次数: 1000
总耗时: 57709.22 ms
平均 GFLOPS: 27127.65
(modelscope) [root@localhost Torch]# CUDA_VISIBLE_DEVICES=0 python GEMM.py
设备: cuda
数据类型: torch.float32
矩阵维度: 9216x9216 * 9216x9216 = 9216x9216
测试次数: 1000
总耗时: 45625.09 ms
平均 GFLOPS: 34312.61
(modelscope) [root@localhost Torch]# CUDA_VISIBLE_DEVICES=0 python GEMM.py
设备: cuda
数据类型: torch.float16
矩阵维度: 9216x9216 * 9216x9216 = 9216x9216
测试次数: 1000
总耗时: 14687.19 ms
平均 GFLOPS: 106590.53
```



理论算力计算: TFLOPS (NVIDIA-L20) = (CUDA 核心数 × 每周期执行的浮点运算次数 × GPU 基础主频 (GHz)) ÷ 1000 = 11776 × 2 × 2.52 ÷ 1000 = 59.35 TFLOPS

L20 核心硬件参数

GPU 架构: NVIDIA Ada Lovelace

制造工艺: 5nm

CUDA 核心数: 11,776

Tensor Core: 368 个（第四代，支持 FP8/FP16/INT8 等）

RT Core: 92 个（第三代）

基础频率 / 加速频率: 1440 MHz / 2520 MHz

显存: 48GB GDDR6 ECC（支持错误校正，提升稳定性）

显存位宽: 384-bit

显存带宽: 864 GB/s

L2 缓存: 96MB

接口: PCIe 4.0 x16（双槽全高全长）

GPU 频率: 1440 MHz

TDP（热设计功耗）: 275W（部分配置下可达 300W）

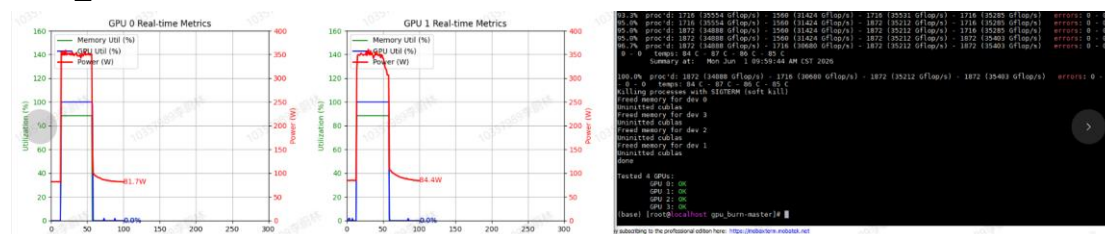
NVIDIA 标称理论算力

型号	CUDA核心	显存类型/容量	显存带宽	FP32性能	INT8推理性能	功耗	架构
L20	11776	GDDR6 48GB	864GB/s	59.8 TFLOPS	362 TOPS	275W	Ada Lovelace
A10	6912	GDDR6 24GB	600GB/s	31.2 TFLOPS	500 TOPS	150W	Ampere
V100	5120	HBM2 16/32GB	900GB/s	15.7 TFLOPS	250 TOPS	300W	Volta
T4	2560	GDDR6 16GB	320GB/s	8.1 TFLOPS	65 TOPS	70W	Turing
P100	3584	HBM2 16GB	720GB/s	9.3 TFLOPS	不支持	250W	Pascal
P4	2560	GDDR5 8GB	224GB/s	5.5 TFLOPS	22 TOPS	50-75W	Pascal

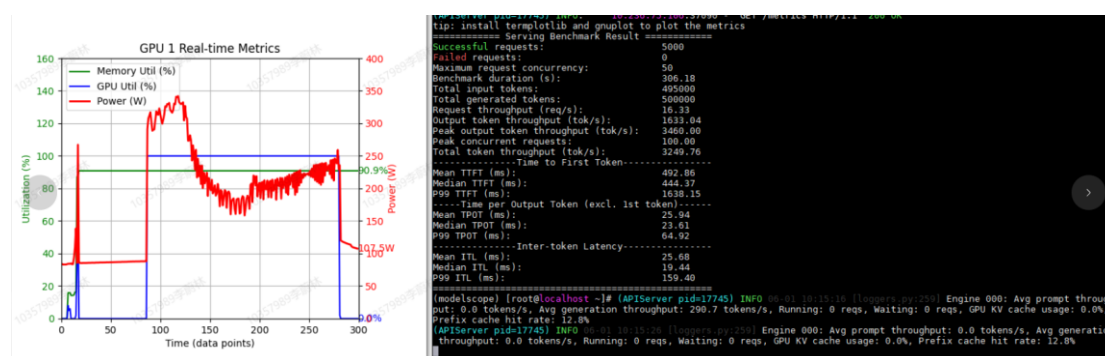
六、结论与展望

6.1 GPU BURN 压测与推理业务对比

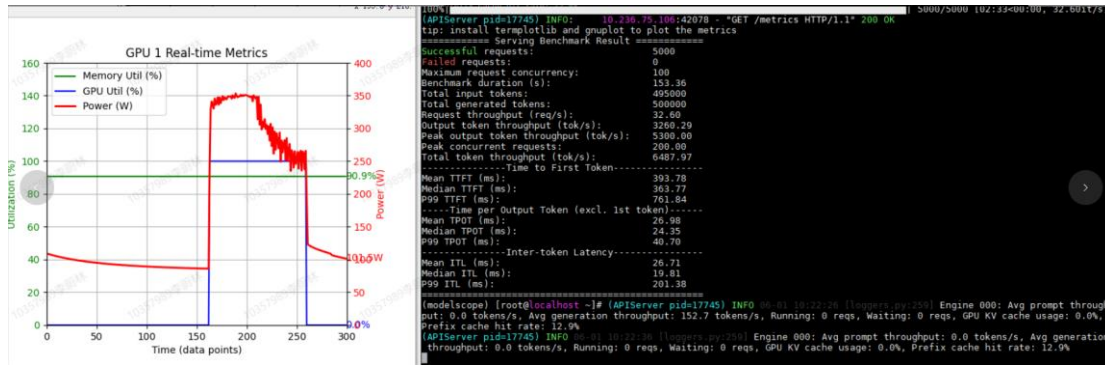
GPU_BURN 压测



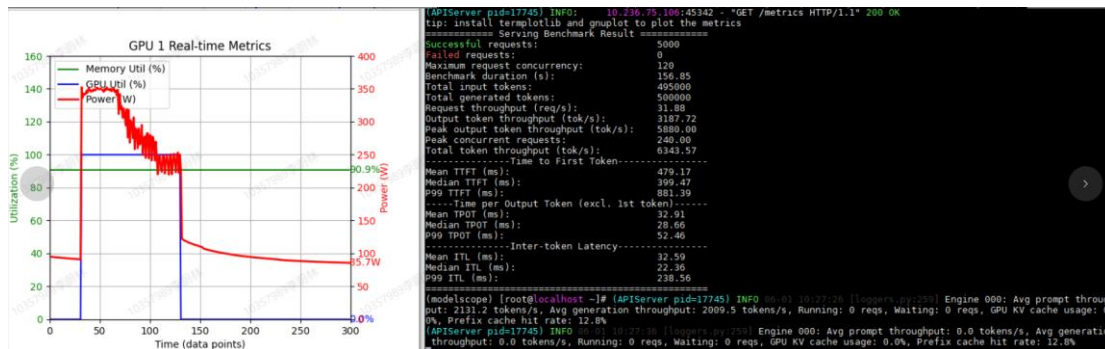
LLM 部署基准测试 1，并发数=50



LLM 部署基准测试 2，并发数=100



LLM 部署基准测试 3，并发数=120



6.2 收益及新增测试用例

	验证场景	测试项	测试用例设计	效果
1	超TDP+整机极限功耗	1、GPU_BURN+混合压力 2、高并发推理业务		待验证
2	动态负载	1、LLM部署、高并发推理业务 2、卸载		待验证
3	异常波动	反复执行。		

6.3 后续计划

- 1、动态负载、异常波动测试，效果验证
- 2、LLM 部署及推理、计算等流程移植到 E200 模组，验证和拦截坏卡。